



HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Introduction to Cryptography and Security

The RSA Cryptosystem

1 The RSA Cryptosystem

2 Encryption and Decryption: Fast Exponentiation

Review: arithmetic mod composites

Let $N = p \cdot q$ where p, q are prime.

$$\mathbb{Z}_N = \{0, \dots, N-1\} \quad ; \quad \mathbb{Z}_N^* = \{ \text{invertible elements in } \mathbb{Z}_N \}$$

Fact

- $x \in \mathbb{Z}_N$ is invertible *iff* $\gcd(x, N) = 1$
- Number of elements in $\mathbb{Z}_N^*$ is

$$\varphi(N) = (p-1)(q-1) = N - p - q + 1$$

Theorem (Euler)

$$\forall x \in \mathbb{Z}_N^* : \quad x^{\varphi(N)} = 1 \pmod{N}$$

First published: Scientific American, Aug. 1977.

Very widely used:

- SSL/TLS: certificates and key-exchange
- Secure e-mail and file systems

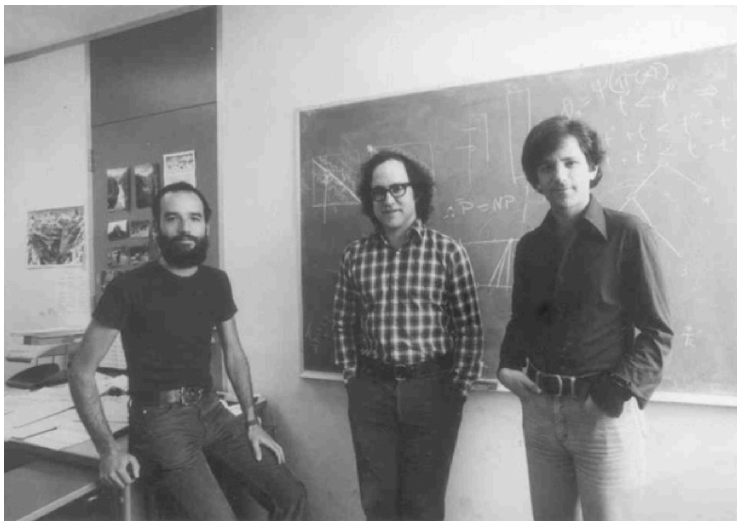


Figure: Adi Shamir, Ron Rivest, and Leonard Adleman

Review: Public key encryption

Definition

A **Public key encryption system** is a triple of algorithms

$$(G, E, D)$$

where:

$G()$: **randomized** algorithm output a key pair (pk, sk) .

$E(pk, m)$: **randomized** algorithm that takes $m \in M$ and output $c \in C$.

$D(sk, c)$: **deterministic** algorithm that takes $c \in C$ and output $m \in M$ or $\perp$ (error).

Consistency: For all (pk, sk) output by G :

$$\forall m \in M : D(sk, E(pk, m)) = m.$$

Key gen $G()$

- choose random primes $p, q \approx 1024$ bits. Set $N = p \cdot q$.
- choose integers e, d such that $e \cdot d = 1 \pmod{\varphi(N)}$
- output $pk = (N, e), \quad sk = (N, d)$

Encryption & Decryption

$E(pk, x) :$

return $x^e \pmod{N}$

$D(sk, y) :$

return $y^d \pmod{N}$

Since $e \cdot d \equiv 1 \pmod{\varphi(N)}$, there exists an integer j such that:

$$e \cdot d = 1 + j \cdot \varphi(N)$$

Therefore:

$$\begin{aligned}(x^e)^d &= x^{1+j \cdot \varphi(N)} \\ &= x \cdot (x^{\varphi(N)})^j \\ &= x \cdot 1 \pmod{N} \\ &= x \pmod{N}\end{aligned}$$

Key Generation in Detail

Output: public key: $pk = (N, e)$ and private key: $sk = (N, d)$

- 1 Choose two large primes p and q .
- 2 Compute $N = p \cdot q$.
- 3 Compute $\varphi(N) = (p - 1)(q - 1)$.
- 4 Select the public exponent $e \in \{1, 2, \dots, \varphi(N) - 1\}$ such that $\gcd(e, \varphi(N)) = 1$.
- 5 Compute the private key d such that $d \cdot e \equiv 1 \pmod{\varphi(N)}$.

- Practical RSA parameters are much, much larger.
- It is recommended that the RSA modulus N be at least 2048 bits long for practical security.
- This results in a bit length for p and q of 1024.

RSA Parameters Example

Prime p :

E0DFD2C2A288ACEBC705EFAB30E4447541A8C5A47A37185C5A9CB98389CE4DE1
9199AA3069B404FD98C801568CB9170EB712BF10B4955CE9C9DC8CE6855C6123

Prime q :

EBE0FCF21866FD9A9F0D72F7994875A8D92E67AEE4B515136B2A778A8048B149
828AEA30BDOBA34B977982A3D42168F594CA99F3981DDABFAB2369F229640115

Public Exponent e :

40B028E1E4CCF07537643101FF72444A0BE1D7682F1EDB553E3AB4F6DD8293CA
1945DB12D796AE9244D60565C2EB692A89B8881D58D278562ED60066DD8211E6
7315CF89857167206120405B08B54D10D4EC4ED4253C75FA74098FE3F7FB751F
F5121353C554391E114C85B56A9725E9BD5685D6C9C7EED8EE442366353DC39

The RSA assumption

RSA is one-way permutation

RSA assumption

For all efficient algorithms A :

$$\Pr[A(N, e, y) = y^{1/e}] < \text{negligible}$$

where $p, q \leftarrow \$$ n-bit primes, $N = pq$, and $y \leftarrow \$ \mathbb{Z}_N^*$

- The security of RSA is based on the hardness of factoring large composite numbers.
- The RSA assumption is that it is hard to compute the e -th root of a random element in $\mathbb{Z}_N^*$.
- The RSA assumption is not known to be equivalent to the hardness of factoring.

Let the two primes $p = 41$ and $q = 17$ be given as setup parameters for RSA.

- 1 Which of the parameters $e_1 = 32$, $e_2 = 49$ is a valid RSA exponent? Justify your choice.
- 2 Compute the corresponding private key $sk = (p, q, d)$. Use the extended Euclidean algorithm for the inversion and show every calculation step.

Encrypt and decrypt by means of the RSA algorithm with the following system parameters:

① $p = 3, q = 11, d = 7, x = 5$

② $p = 5, q = 11, e = 3, y = 9$

Only use a pocket calculator at this stage.

1 The RSA Cryptosystem

2 Encryption and Decryption: Fast Exponentiation

Example

Suppose we want to compute

$$3^{218} \pmod{1000}.$$

First, we write 218 in base 2:

$$218 = 2 + 2^3 + 2^4 + 2^6 + 2^7.$$

Thus, 3^{218} becomes

$$3^{218} = 3^{2+2^3+2^4+2^6+2^7} = 3^2 \cdot 3^{2^3} \cdot 3^{2^4} \cdot 3^{2^6} \cdot 3^{2^7}.$$

Notice that it is easy to compute the powers

$$3, 3^2, 3^{2^2}, 3^{2^3}, 3^{2^4}, \dots$$

Example (continued)

We create a table

i	0	1	2	3	4	5	6	7
$3^{2^i} \pmod{1000}$	3	9	81	561	721	841	281	961

then compute

$$\begin{aligned}3^{2^{18}} &= 3^2 \cdot 3^{2^3} \cdot 3^{2^4} \cdot 3^{2^6} \cdot 3^{2^7} \\&\equiv 9 \cdot 561 \cdot 721 \cdot 281 \cdot 961 \pmod{1000} \\&\vdots \\&\equiv 489 \pmod{1000}.\end{aligned}$$

Example (Square-and-Multiply for x^{26})

$$26 = \langle 11010 \rangle_2 = \langle h_4 h_3 h_2 h_1 h_0 \rangle_2$$

#	Operation	Result	Note
0	$x = x^1$	initial	$h_4 = 1$
1a	$(x^1)^2 = x^2$	x^{10_2}	SQ, bit processed: h_3
1b	$x^2 \cdot x = x^3$	x^{11_2}	MUL, since $h_3 = 1$
2a	$(x^3)^2 = x^6$	x^{110_2}	SQ, bit processed: $h_2 = 0$
2b			no MUL, since $h_2 = 0$
3a	$(x^6)^2 = x^{12}$	x^{1100_2}	SQ, bit processed: h_1
3b	$x^{12} \cdot x = x^{13}$	x^{1101_2}	MUL, since $h_1 = 1$
4a	$(x^{13})^2 = x^{26}$	x^{11010_2}	SQ, bit processed: h_0
4b			no MUL, since $h_0 = 0$

Fast Computation of $a^b \pmod n$

MODULAR-EXPONENTIATION(a, b, n)

$c = 0$

$d = 1$

Represent $b = \langle b_k, b_{k-1}, \dots, b_0 \rangle_2$

for $i = k$ **downto** 0

$c = 2c$

$d = (d \cdot d) \pmod n$

if $b_i == 1$ **then**

$c = c + 1$

$d = (d \cdot a) \pmod n$

return d

- The value of c is $\langle b_k, b_{k-1}, \dots, b_{i+1} \rangle_2$ and $d = a^c \pmod n$.

Example

Compute $7^{560} \bmod 561$

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
d	7	49	157	526	160	241	298	166	67	1

- Result of computing $a^b \pmod n$ with

$$a = 7, \quad b = 560 = \langle 1000110000 \rangle_2, \text{ and } n = 561$$

- The value of c is $\langle b_k, b_{k-1}, \dots, b_{i+1} \rangle_2$ and $d = a^c \bmod n$.
- The final result is 1

Recursive Algorithm for $a^b \bmod n$

MODULAR-EXPONENTIATION(a, b, n)

if $b == 0$ **then return** 1

if $b == 1$ **then return** a

$r = \text{MODULAR-EXPONENTIATION}(a, b/2, n)$

$r = r * r$

if $b \bmod 2 == 1$ **then** $r = r * a$

return r

Computing modular exponentiation efficiently is central to using RSA in practice. Compute the following exponentiations $x^e \bmod m$ using the square-and-multiply algorithm:

- ① $x = 2, e = 79, m = 101$
- ② $x = 3, e = 197, m = 101$
- ③ $x = 5, e = 54, m = 151$
- ④ $x = 8, e = 127, m = 151$

After every iteration step, show the exponent of the intermediate result in binary notation.

Exercise

- Given that you know $\varphi(n)$,
- explain how to compute the modular inverse

$$a^{-1} \pmod n \quad \text{for any } a \in \mathbb{Z}_n^*$$

using the MODULAR-EXPONENTIATION algorithm.

Hint: Recall that $a^{\varphi(n)} = 1 \pmod n$.

- Modular exponentiation is computationally intensive
- Even with the square-and-multiply algorithm, RSA can be quite slow on constrained devices such as smart cards
- Some important tricks:
 - Short public exponent e
 - Chinese Remainder Theorem (CRT)
 - Exponentiation with pre-computation (not covered here)

Fast Encryption with Small e

- Fast encryption with small public exponent
 - Choosing a small public exponent e does not weaken the security
 - A small public exponent improves the speed of the RSA encryption
- Public Key e as binary string #MUL + #SQ

Exponent	Binary	#MUL + #SQ
$2^1 + 1$	$(11)_2$	$1 + 1 = 2$
$2^4 + 1$	$(10001)_2$	$4 + 1 = 5$
$2^{16} + 1$	$(10000000000000001)_2$	$16 + 1 = 17$

- This is a commonly used trick (e.g., SSL/TLS, etc.) and makes RSA the fastest asymmetric scheme with regard to encryption!

Malleability

- Given a ciphertext $c = E(pk, m) = m^e \bmod N$, an attacker can forge a new ciphertext $E(pk, m^a) = c^a \bmod N$ for any a .

Chosen Ciphertext Attack

- Given a ciphertext $c = E(pk, m)$ for unknown m , the attacker can ask for

$$D(sk, c \cdot a^e \bmod N) = c^d \cdot a^{ed} = m \cdot a = m'$$

and compute $m = m' \cdot a^{-1} \bmod N$.

Padding: RSA PKCS #1 v1.5

To prevent attacks, RSA should be used with padding schemes.
PKCS #1 v1.5 is a widely used padding standard

Padding Format

$$m = 00\ 02\ [\text{random padding string}]\ 00\ [\text{data}]$$

Encryption Process

- 1 Encrypter pads the message according to PKCS #1 v1.5 format
- 2 Then encrypts the padded message using RSA public key

Decryption Process

- 1 Decrypter decrypts using RSA private key
- 2 Strips off padding to recover original data

Real-world Implementation Choices for RSA

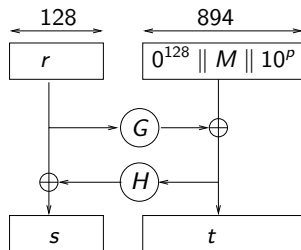
- Most of the internet has moved to at least 2048-bit keys
- Nearly everyone uses $e = 65537$. Almost nobody uses $e > 32$ bits
- RSA key exchange supported by default for TLS
- PKCS#1v1.5 is universally used

Receiver keys: $pk = (N, e)$ and $sk = (N, d)$ where $|N| = 1024$

Hash functions: $G: \{0, 1\}^{128} \rightarrow \{0, 1\}^{894}$ and $H: \{0, 1\}^{894} \rightarrow \{0, 1\}^{128}$

Algorithm $\mathcal{E}_{N,e}(M)$ // $|M| \leq 765$

$r \xleftarrow{\$} \{0, 1\}^{128}; p \leftarrow 765 - |M|$



$x \leftarrow s || t$

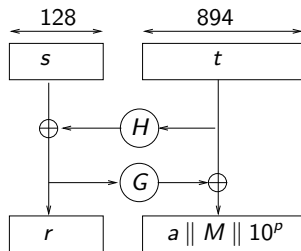
$C \leftarrow x^e \bmod N$

return C

Algorithm $\mathcal{D}_{N,d}(C)$ // $C \in \mathbb{Z}_N^*$

$x \leftarrow C^d \bmod N$

$s || t \leftarrow x$



if $a = 0^{128}$ **then return** M

else return $\perp$

Protocols:

- SSL ver. 2.0, 3.0 / TLS ver. 1.0, 1.1
- SSH ver 1.0, 2.0
- ...

Standards:

- RSA PKCS #1 versions 1.5, 2.0
- IEEE P1363
- NESSIE (Europe)
- CRYPTREC (Japan)
- ...

- RSA is the most widely used public-key cryptosystem
- RSA is mainly used for key transport and digital signatures
- The public key e can be a short integer, the private key d needs to have the full length of the modulus n
- RSA relies on the fact that it is hard to factorize n
- Currently 1024-bit cannot be factored, but progress in factorization could bring this into reach within 10-15 years. Hence, RSA with a 2048 or 3076 bit modulus should be used for long-term security
- A naïve implementation of RSA allows several attacks, and in practice RSA should be used together with padding



25
SOICT

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Thank you!



soict.hust.edu.vn/



fb.com/groups/soict

